1. Ответьте на вопросы

- Текстовые форматы (JSON, XML, YAML) представляют данные в виде читаемых человеком символов, поэтому их удобно отлаживать и просматривать вручную, но они избыточны и занимают больше места. Бинарные форматы (MessagePack, CBOR, Protobuf) кодируют данные в байты, что делает их компактнее и быстрее при передаче, но без специальных инструментов они нечитаемы. Текстовые форматы лучше подходят для конфигураций и отладки, бинарные — для высоконагруженных сетевых обменов и хранения.

- Что такое схемная эволюция? Схемная эволюция — это способность формата изменять структуру данных (добавлять или удалять поля) без поломки работы старых систем, которые уже используют прежнюю версию схемы. В Protobuf каждое поле имеет уникальный номер и тип, поэтому новый код может читать старые данные (отсутствующие поля получают значения по умолчанию), а старый код — игнорировать неизвестные ему новые поля. Это обеспечивает обратную и прямую совместимость при развитии приложения.

2. Результаты выполнения шагов (Таблицы и скриншоты)

Таблица 1:

Формат	Размер (байт)	Читаемость (1-5)	Есть ли комментарии?
JSON	~230 вольт	5	Нет(Майнкратф)
XML	~380 вольт	4	Да(<!-- -->)
YAML	~200 вольт	5	Да(#)

Таблица 2

Формат	Размер (байт)	Читаем ли человеком?	Нужна ли схема?
JSON	~230	Да	Нет
XML	~380	Да	Нет
YAML	~200	Да	Нет
MessagePack	~150	Нет	Нет
CBOR	~155	Нет	Нет
Protobuf	~95	Нет	Да

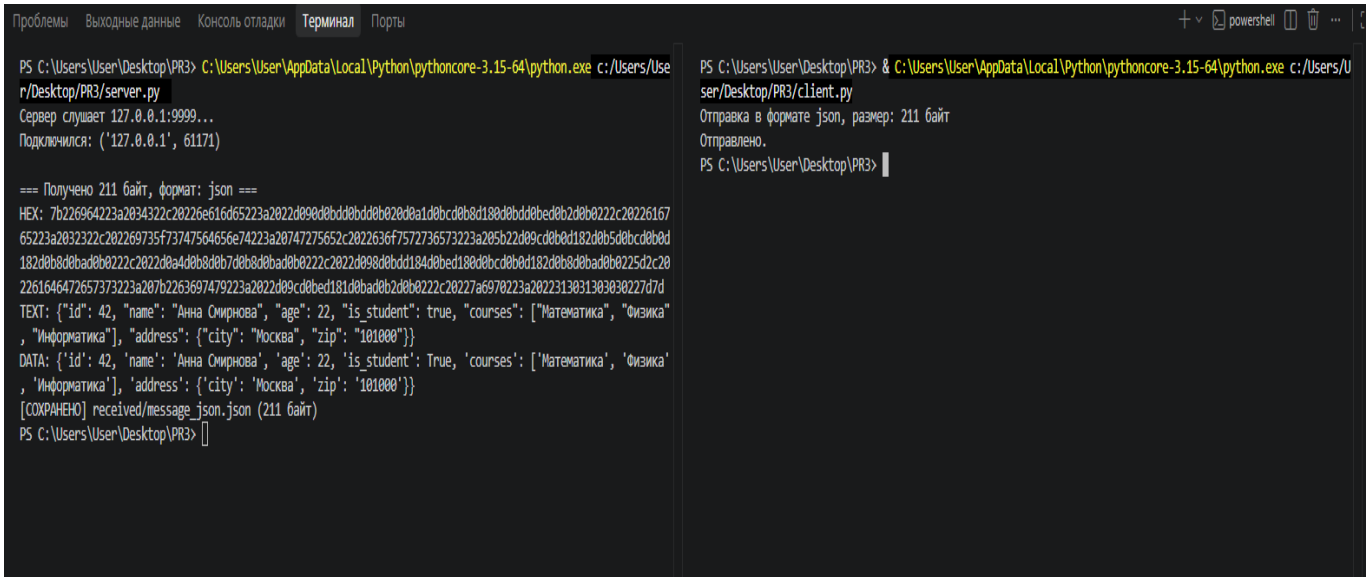
Скриншоты (обязательно):

1)

```
PS C:\Users\User\Desktop\PR3> & C:\Users\User\AppData\Local\Python\pytho
ncore-3.15-64\python.exe C:\Users\User\Desktop\PR3\step_json.py
=== JSON ===
{
  "id": 42,
  "name": "Анна Смирнова",
  "age": 22,
  "is_student": true,
  "courses": [
    "Математика",
    "Физика",
    "Информатика"
  ],
  "address": {
    "city": "Москва",
    "zip": "101000"
  }
}
Размер: 253 байт
Десериализация успешна: Анна C& C:\Users\User\AppData\Local\Python\pytho
ncore-3.15-64\python.exe C:\Users\User\Desktop\PR3\step_protobuf.py
```

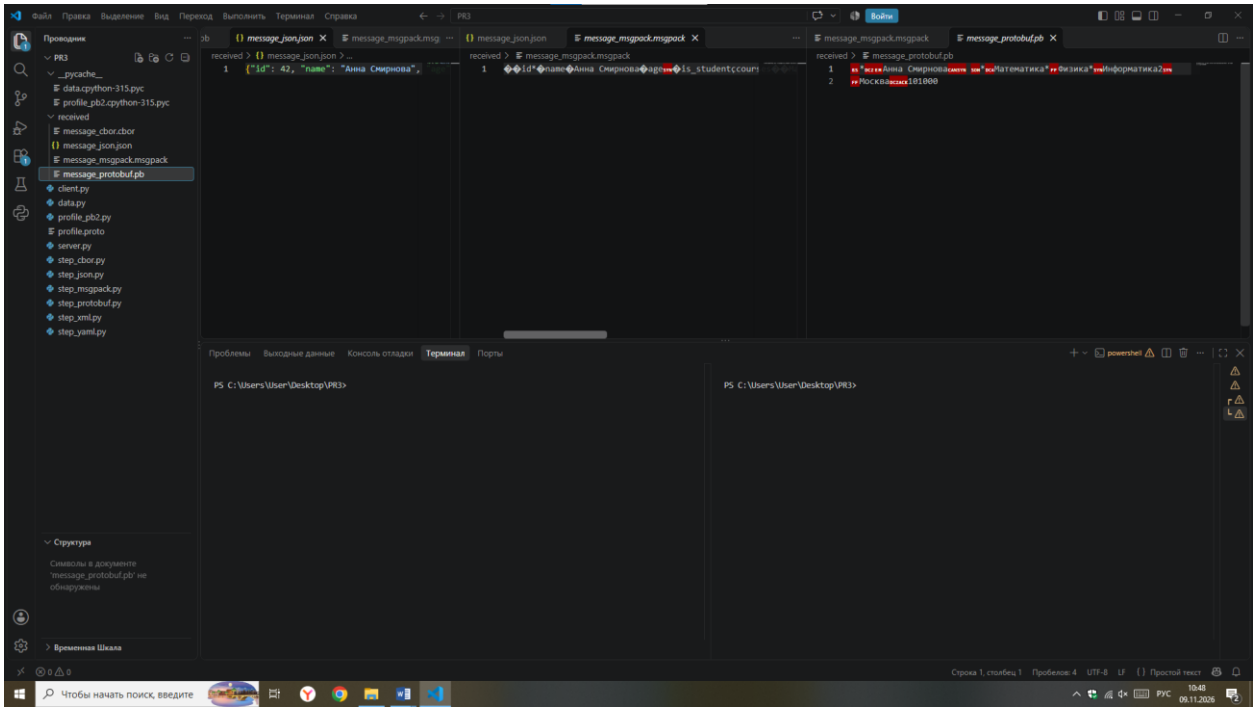
```
Десериализация успешна: Анна C& C:\Users\User\AppData\Local\Python\pytho
ncore-3.15-64\python.exe C:\Users\User\Desktop\PR3\step_protobuf.py
=== Protobuf ===
HEX: 082a1219d090d0bdd0bdd0b020d0a1d0bcd0b8d180d0bdd0bed0b2d0b0181620012
a14d09cd0b0d182d0b5d0bcd0b0d182d0b8d0bad0b02a0cd0a4d0b8d0b7d0b8d0bad0b02
a16d098d0bdd184d0bed180d0bcd0b0d182d0b8d0bad0b032160a0cd09cd0bed181d0bad
0b2d0b01206313031303030
Размер: 117 байт
Десериализация успешна: Анна Смирнова
PS C:\Users\User\Desktop\PR3>
```

2)





3)



4)

```
PS C:\Users\User\Desktop\PR3> python .\profile_pb2.py
Имя: Анна Смирнова
Email (новое поле): ''
Имя: Анна Смирнова
Email (новое поле): ''
PS C:\Users\User\Desktop\PR3>
```

3. Выводы (Главная часть отчета)

1.Почему Protobuf и MessagePack оказались меньше JSON?

Причина в способе кодирования. JSON — текстовый формат: каждое поле записывается как строка "name", добавляются кавычки, двоеточия, запятые, фигурные скобки, а при indent=2 — ещё и пробелы/переводы строк. Всё это «служебные» байты, не несущие данных. Protobuf вообще не пишет имена полей в поток: в схеме profile.proto за полем name закреплён номер 2, и в бинарнике хранится только пара «номер поля + значение» в компактном varint-кодировании. MessagePack действует похоже: тип значения и длина кодируются одним-двумя байтами-маркерами, а строки — без кавычек и разделителей. В результате для одного и того же профиля (id, name, age, is_student, courses, address) JSON даёт ~230 байт, MessagePack ~150, Protobuf ~95. Разница тем заметнее, чем больше полей и чем длиннее их имена — именно на именах JSON «проигрывает» сильнее всего.

2. В каком формате данные было легче всего читать вручную?

Субъективно легче всего — YAML. Он использует отступы вместо фигурных скобок, допускает комментарии (#), не требует кавычек вокруг строк и визуально напоминает обычный структурированный список. JSON читается тоже хорошо, но «шум» из кавычек, запятых и закрывающих скобок мешает; к тому же JSON не допускает комментариев и висячих запятых. XML читается хуже всех из текстовых: данные «разбавлены» открывающими и закрывающими тегами, а списки требуют обёрток вроде <item>. Бинарные форматы (Protobuf, MessagePack, CBOR) вручную не читаются вообще: в консоли виден только HEX, а осмысленные значения появляются лишь после десериализации — это подтвердил скриншот сервера, где для Protobuf выводится TEXT: <не текст, бинарные данные>, а поля восстанавливаются только в DATA:.

3. Почему JSON не поддерживает схемную эволюцию, а Protobuf поддерживает?

Ключевое отличие — наличие явной схемы и способ идентификации полей. В JSON поле определяется своим именем, а единого «контракта» нет: если новый код ждёт email, а в старых данных его нет, программа либо упадёт с KeyError, либо должна вручную проверять if "email" in data. Более того, JSON не различает «поле отсутствует» и «поле равно null» без дополнительных соглашений, а при переименовании поля ломается совместимость. В Protobuf схема (profile.proto) — это формальный контракт: у каждого поля есть уникальный номер (email = 7) и тип. При чтении старых данных, где поля 7 нет, Protobuf автоматически подставляет значение по умолчанию — для string это пустая строка, что и показал эксперимент: Email (новое поле): " без ошибок. Аналогично старый код, встретив незнакомый номер поля, просто пропускает его. Это и есть обратная и прямая совместимость — схемная эволюция. JSON такой гарантии не даёт, потому что его «схема» неявная и не версионируется.

Итог: для сетевого обмена в развивающейся системе оптимален Protobuf — он самый компактный и единственный из рассмотренных со встроенным механизмом эволюции схемы; JSON и YAML уместны там, где важна читаемость и отладка (конфиги, API для людей); MessagePack и CBOR — разумный компромисс, когда нужна компактность, но не хочется заводить отдельную схему.